Intent Distribution based Bipartite Graph Representation Learning

Haojie Li
lihaojie@qust.edu.cn
School of Data Science, Qingdao University of Science and Technology
Qingdao, China

Wei Wei
weiwei961112@163.com
School of Data Science, Qingdao University of Science and Technology
Qingdao, China

Guanfeng Liu
guanfeng.liu@mq.edu.au
School of Computing, Macquarie University
Sydney, Australia

Jinhuan Liu
liujinhuan.sdu@gmail.com
School of Data Science, Qingdao University of Science and Technology
Qingdao, China

Feng Jiang
jiangkong2002@163.com
School of Information Science and Technology, Qingdao University of Science and Technology
Qingdao, China

Junwei Du*
djwqd@163.com
School of Data Science, Qingdao University of Science and Technology
Qingdao, China

ABSTRACT

Bipartite graph representation learning embeds users and items into a low-dimensional latent space based on observed interactions. Previous studies mainly fall into two categories: one reconstructs the structural relations of the graph through the representations of nodes, while the other aggregates neighboring node information using graph neural networks. However, existing methods only explore the local structural information of nodes during the learning process. This makes it difficult to represent the macroscopic structural information and leaves it easily affected by data sparsity and noise. To address this issue, we propose the Intent Distribution based Bipartite graph Representation learning (IDBR) model, which explicitly integrates node intent distribution information into the representation learning process. Specifically, we obtain node intent distributions through clustering and design an intent distribution based graph convolution neural network to generate node representations. Compared to traditional methods, we expand the scope of node representations, enabling us to obtain more comprehensive representations of global intent. When constructing the intent distributions, we effectively alleviated the issues of data sparsity and noise. Additionally, we enrich the representations of nodes by integrating potential neighboring nodes from both structural and semantic dimensions. Experiments on the link prediction and recommendation tasks illustrate that the proposed approach outperforms existing state-of-the-art methods. The code of IDBR is available at https://github.com/rookitkitlee/IDBR.

*Junwei Du is the corresponding author.

SIGIR '24, July 14–18, 2024, Washington, DC, USA

ACM ISBN 979-8-4007-0431-4/24/07
https://doi.org/10.1145/3626772.3657739

CCS CONCEPTS

• Information systems → Information retrieval; **Recommender systems**; **• Computing methodologies → Neural networks**.

KEYWORDS

Bipartite Graph, Intent Distribution, Recommendation, Link Prediction

ACM Reference Format:
Haojie Li, Wei Wei, Guanfeng Liu, Jinhuan Liu, Feng Jiang, and Junwei Du. 2024. Intent Distribution based Bipartite Graph Representation Learning . In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24), July 14–18, 2024, Washington, DC, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3626772.3657739

1 INTRODUCTION

Bipartite graphs, serving as a universal storage framework, can efficiently capture the interaction information between two different classes of nodes. The network representation-based methods, owing to their effectiveness in capturing the structural information of nodes, have become the mainstream methods for analyzing problems in bipartite graph contexts [14, 20, 39]. They are widely used in various fields, including recommendation tasks [5, 9, 42], link prediction [23, 25], node classification [15] and drug development [11].

Current bipartite graph representation learning methods primarily fall into two categories: relation reconstruction [12, 28, 43] and neighborhood aggregation [1, 14, 17, 39]. Relation reconstruction aims to reconstruct the structural relations between nodes using their node representations. For instance, CSE [5] mines direct interactions among heterogeneous nodes and implicit interactions among homogenous nodes in bipartite graphs, and uses the representations of nodes to reconstruct these relations. Gao [9] improves sampling strategies in relation reconstruction by using a random walk method that aligns with bipartite graph node distributions, allowing for better understanding and exploration of the graph's structural relations. With the advancement of deep learning, neighborhood aggregation methods have emerged, which generate node representations by iteratively integrating information from neighboring nodes [2, 6, 18, 40]. For example, the Graph Convolutional Network (GCN) [45] aggregates information from each node and its neighbors via a convolution process to generate new node representations. This approach extracts high-order semantic information from the nodes through multi-layer convolution. HiGNN [22], alternating between multiple Graph Neural Network (GNN) blocks and deterministic clustering algorithms, effectively learns node representations by considering the hierarchical relations and group characteristics between users and items.

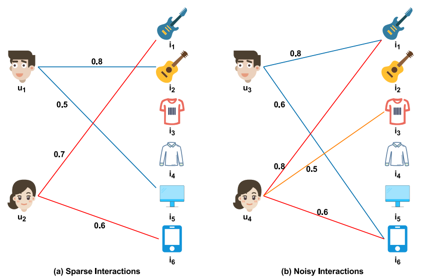

Figure 1: The sparse and noisy interactions.

Although current bipartite graph representation learning methods have achieved notable advancements, they primarily focus on the local structural information of nodes during the learning process. In relation reconstruction methods [12, 35], although expanding the search range of relations broadens the scope of node representations, it carries the risk of over-smoothing by causing nodes to resemble too many neighbors. Similarly, neighborhood aggregation methods [6, 18] using graph neural networks face the challenge of over-smoothing when increasing network layers. Consequently, existing methods are limited to representing only the local structural information of the graph, making it difficult to accurately characterize its macroscopic structure. Additionally, representations based on local structural information are highly susceptible to the impacts of sparsity and noise. For example, consider a bipartite graph as shown in Figure 1(a), although users u_1 and u_2 share common interests, the sparsity of the edges in the graph structure makes it impossible to depict a path from u_1 to u_2, preventing current algorithms from extracting the similarity between them. In addition, considering a bipartite graph shown in Figure 1(b), users u_3 and u_4 should have similar interactions, but an erroneous evaluation of item i_3 by u_4 leads to a significant difference in the representations of u_3 and u_4. In theory, expanding the range of node representations provides a potential solution to these challenges. However, the inherent limitations of current bipartite graph representation algorithms pose substantial obstacles to effectively addressing these issues.

To address the above issues, we propose the Intent Distribution based Bipartite graph Representation learning (IDBR) model. IDBR explicitly integrates the intent distribution information of nodes, enabling a more comprehensive understanding of latent information such as users' interests and categories of items. The intent information of nodes has been confirmed in numerous studies to play a significant role in enhancing the effectiveness of representation learning [4, 7, 26, 30, 32, 36]. Specifically, we first consider the interaction behaviors of nodes as combinations of multiple latent intents. IDBR utilizes the Gaussian Mixture Model [31] to extract combinations of intents from interactions, identifying the cluster centers as intent points. It then aggregates all interactions of a node to form its distinct intent distribution. Subsequently, we construct a graph convolutional neural network based on this intent distribution. In this network, the aggregated information for node representations focuses on pure intent points, rather than neighboring nodes. Different node representations are distinguished by their distribution differences with these intent points. Since intent points are generated through the clustering of all nodes, the IDBR model can effectively obtain the global intent representations of nodes and address the issues of data sparsity and noise. Additionally, we consider the connection strength and distance between nodes, mining the similarities between nodes and their neighbors from both structural and semantic perspectives, and using them as constraints to enrich the representation of nodes. The contributions of our work are summarized as follows:

(1) We propose an intent distribution based graph convolutional neural network approach, wherein the network aggregates more refined intent information instead of neighboring node information. This enables us to efficiently capture the global intent information of nodes. Moreover, it effectively addresses the issues of data sparsity and noise when creating intent distributions.
(2) We explore similarity relations between nodes and their neighbors from both structural and semantic perspectives, using these as constraints to enrich our node representations. Experiments show that in tasks related to the link prediction and recommendation, enriching representation through structural neighbors can improve performance by an average of 6.21%, and enriching representation through semantic neighbors can improve performance by an average of 4.24%.
(3) Experiments in the link prediction and recommendation tasks have illustrated the outperformance of our model. Compared to the best results obtained with existing methods, our method can, on average, enhance performance by 1.30% in the link prediction task and by 9.84% in the recommendation task.

2 PRELIMIARIES

Bipartite Graph: Let $G = \{U, I, E\}$ be a bipartite graph, where U and I are sets of two classes of nodes in G, and $E \subseteq U \times V$ represents the set of edges of G. For each edge $e_{u,i}$ in E, there exists a non-negative number $w_{u,i}$ to denote the connection strength (also called weight) from node u to node i, where $u = 1, 2, ..., |U|$ and $i = 1, 2, ..., |I|$. We use the weight matrix $W = [w_{u,i}]$ to represent all the weights in the bipartite graph.

Bipartite Graph Representation: Bipartite graph representation aims to transform all graph nodes into compact and dense vector representations within a lower-dimensional space. These node representations should effectively capture the structural relations within the graph where the nodes reside. Formally, the problem can be defined as:

- **Input:** A bipartite graph $G = \{U, I, E\}$, and its weight matrix W.

- **Output:** A mapping function $f : U \cup I \rightarrow \mathbb{R}^d$ from nodes in the graph to the lower dimensional space, where d denotes the dimension of the lower dimensional space.

3 METHODOLOGY

3.1 Overview

In this section, we illustrate an overview of IDBR, which can be divided into the following steps as shown in Figure 2.

- **Step 1:** The representations of nodes U (as shown in Figure 2(a)) and V (as shown in Figure 2(b)) are separately clustered, leading to the creation of cluster centers P (as shown in Figure 2(c)) and Q (as shown in Figure 2(d)).
- **Step 2:** Perform the intent distribution based graph convolutional neural network (as shown in Figure 2(e)), and enriching representation through structural neighbors and semantic neighbors (as shown in Figure 2(f) and Figure 2(g)) to generate new node representations for U (as shown in Figure 2(h)) and V (as shown in Figure 2(i)).
- **Step 3:** Repeat steps 1-2 until all epochs are completed.

3.2 Intent Distribution Based Node Representation

3.2.1 Intent distributions of nodes. In this section, we illustrate the process of constructing the intent distributions of nodes. Initially, we employ the Gaussian Mixture (GMM) [31] to cluster nodes of the same class within the bipartite graph G.

$$P, D^U = GMM(U),\ Q, D^I = GMM(V), \tag{1}$$

where P and Q denote the set of cluster centers in U and I, respectively. $D^U \in \mathbb{R}^{|U|\times|P|}$ and $D^I \in \mathbb{R}^{|I|\times|Q|}$ are distribution matrices representing the distributions of nodes in U and I across classes in P and Q, respectively. Then, we use the original weight matrix and node distribution matrices to generate the intent distributions of nodes.

$$I^U = WD^I,\ I^I = W^T D^U, \tag{2}$$

where $I^U \in \mathbb{R}^{|U|\times|Q|}$ and $I^I \in \mathbb{R}^{|I|\times|P|}$ represent the intent distributions of nodes in U and I, respectively. Below, we illustrate how intent distribution solves the problems of data sparsity and noise in local structures.

For a given sparse bipartite graph, as shown in the left half of Figure 3(a), we cluster the item nodes within this bipartite graph, as shown in the right half of Figure 3(a). Then we utilize the original weight matrix and node distribution matrix to generate the intent distributions of users, as shown in Figure 3(b). Although u_1 and u_2 are not directly connected by a path in the original graph, we can uncover their similarity by analyzing their similar intent distributions. In addition, for a given noisy bipartite graph, as shown in the left half of Figure 4(a), we obtained the intent distributions of u_3 and u_4 using the same method as shown in Figure 4(b). Although u_4 has a noisy interaction, the intent distributions of u_3 and u_4, after normalization, yield similar results.

3.2.2 Intent Distribution based Graph Convolutional Neural Network. In IDBR, we utilize GNN to model the interactions between nodes and intent points. Specifically, we follow LightGCN [14] in our propagation function, which involves discarding non-linear activations and feature transformations. Additionally, acknowledging the varied distributions of nodes across different intents, we adjust the weights for node aggregation according to these intent distributions. The specific propagation process is as follows:

$$z_u^{(l+1)} = \sum_{q\in Q} \alpha_{u,q} z_q^{(l)},\ \alpha_{u,q} = \frac{I^U_{u,q}}{\sum_{q'\in Q} I^U_{u,q'}}, \tag{3}$$

$$z_i^{(l+1)} = \sum_{p\in P} \beta_{i,p} z_p^{(l)},\ \beta_{i,p} = \frac{I^I_{i,p}}{\sum_{p'\in P} I^I_{i,p'}}, \tag{4}$$

where l represents the number of propagation layers, z_u represents the vector representation of u, and z_q represents the vector representation of the cluster center q, α and β represent the weights of the cluster centers in the propagation process. Inspired by UltraGCN [27], our goal is to achieve the following convolution state after infinite layers propagation:

$$z_u = \lim_{l\to\infty} z_u^{(l+1)} = \lim_{l\to\infty} z_u^{(l)},\ z_i = \lim_{l\to\infty} z_i^{(l+1)} = \lim_{l\to\infty} z_i^{(l)}. \tag{5}$$

Equation (5) indicates that after aggregation through the intent distribution, the representation of each node is equivalent to itself. We denote the final aggregated representation of the user u(or the item i) as z_u (or z_i). The Equations (3) and (4) can then be rewritten as follows:

$$z_u = \sum_{q\in Q} \alpha_{u,q} z_q,\ z_i = \sum_{p\in P} \beta_{i,p} z_p. \tag{6}$$

After obtaining the final representations of nodes, we use the inner product to predict the likelihood of interaction between the user u and the item i:

$$\hat{y}_{u,i} = z_u^T z_i. \tag{7}$$

We employ two constraints to ensure that our representation based on intent distribution can adequately represent node features. First, we use the reconstructed constraint $\mathcal{L}_R$ to ensure that the model can reconstruct the original bipartite graph structure. To reconstruct the original bipartite graph structure, we focus on minimizing the empirical distribution of the co-occurrence probabilities of nodes and the reconstructed distribution via node representations, which has been successfully applied in LINE[12], BiNE[10], etc. The empirical probability distribution of two connected nodes u and i in the bipartite network is defined as:

$$Y_{u,i} = \frac{w_{u,i}}{\sum_{e_{s,t}\in E} w_{s,t}}, \tag{8}$$

where $w_{s,t}$ is the weight of edge $e_{s,t}$. The reconstructed distribution of nodes u and i via node representations is:

$$\hat{Y}_{u,i} = \frac{1}{1 + exp(-\hat{y}_{u,i})}. \tag{9}$$

Then, we adopt the KL divergence to learn the node representations by minimizing the difference between the above two distributions. Thus, we define the reconstructed constraint $\mathcal{L}_R$ as follows:

$$minimize\ \mathcal{L}_R = KL(Y||\hat{Y}) = \sum_{Y_{u,i}\in Y} [Y_{u,i} log \frac{Y_{u,i}}{\hat{Y}_{u,i}}]. \tag{10}$$

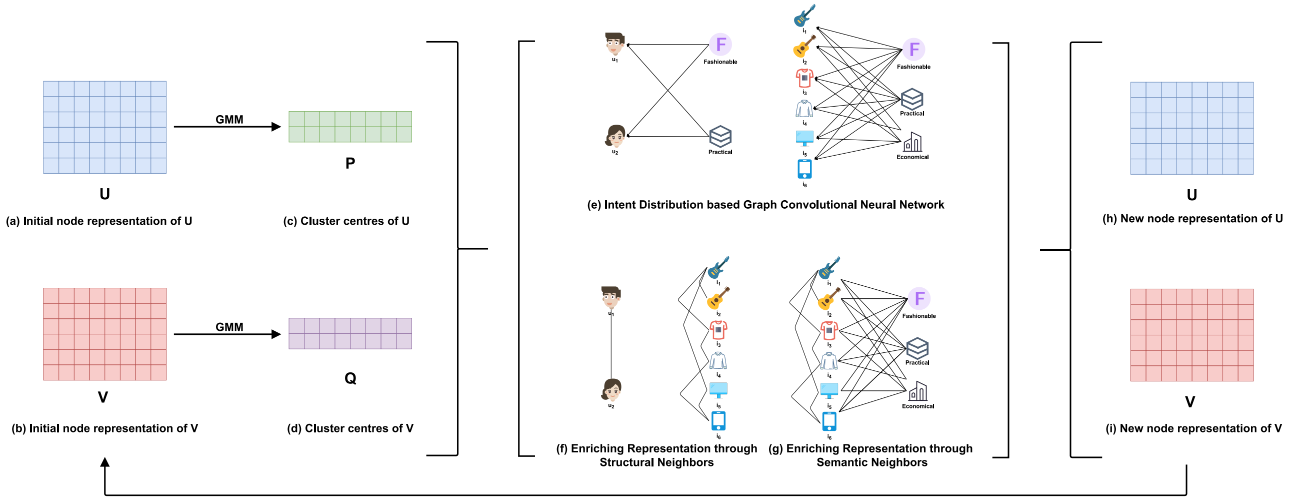

Figure 2: An overview of IDBR.

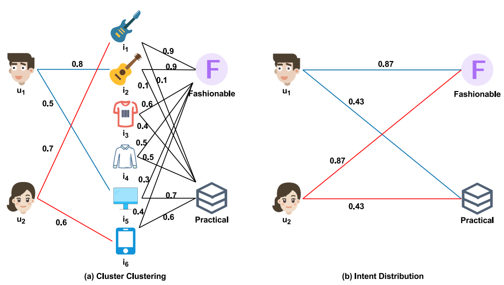

Figure 3: Intent distribution on sparse data.

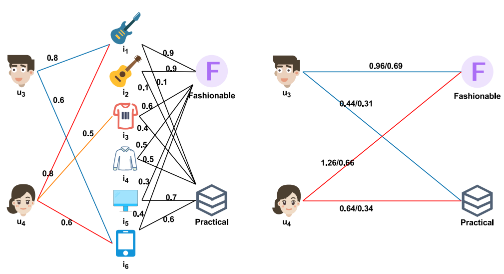

Figure 4: Intent distribution on noisy data.

Furthermore, we implemented a constraint $\mathcal{L}_I$ to stabilize node representations in our model after infinite layers propagation. Specifically, we replace Equation (7) with Equation (11):

$$\hat{y}^U_{u,i} = \sum_{q \in Q} \alpha_{u,q} z_q^T z_i, \;\; \hat{y}^I_{u,i} = \sum_{p \in P} \beta_{i,p} z_u^T z_p. \tag{11}$$

Thus, we have obtained two reconstructed distributions, $\hat{Y}^U_{u,i}$ and $\hat{Y}^I_{u,i}$, which have undergone the propagation of intent distribution, where

$$\hat{Y}^U_{u,i} = \frac{1}{1 + exp(-\hat{y}^U_{u,i})}, \;\; \hat{Y}^I_{u,i} = \frac{1}{1 + exp(-\hat{y}^I_{u,i})}. \tag{12}$$

We align the node's reconstructed distribution pre- and post-intent distribution propagation by minimizing the KL divergence to ensure its stable representation.

$$minimize \;\; \mathcal{L}^U_I = KL(\hat{Y}||\hat{Y}^U) = \sum_{\hat{Y}_{u,i} \in \hat{Y}} [\hat{Y}_{u,i} log \frac{\hat{Y}_{u,i}}{\hat{Y}^U_{u,i}}], \tag{13}$$

$$minimize \;\; \mathcal{L}^I_I = KL(\hat{Y}||\hat{Y}^I) = \sum_{\hat{Y}_{u,i} \in \hat{Y}} [\hat{Y}_{u,i} log \frac{\hat{Y}_{u,i}}{\hat{Y}^I_{u,i}}]. \tag{14}$$

The complete infinite layers propagation constraint $\mathcal{L}_I$ is the sum of the above two losses.

$$minimize \;\; \mathcal{L}_I = \mathcal{L}^U_I + \mathcal{L}^I_I. \tag{15}$$

3.3 Enriching representation through neighbors

3.3.1 Enriching Representation through Structural Neighbors. Inspired by NCL [24], we constrain the relations between nodes and their homogeneous neighbors, aiming to capture the latent connections between users and items. The rationale behind this approach is based on the assumption that nodes sharing the same neighbors tend to have higher similarity. In NCL [24], the alignment between homogeneous neighbor nodes is achieved through the InfoNCE [29]. However, this method does not consider the differences in connection strength in weighted bipartite graphs, nor does it consider the proximity between nodes at different hop distances. Therefore, a more refined and precise method of node pair constraints is needed.

In our work, we initially construct user-user and item-item co-occurrence graphs by linking nodes that appear together. This process leads to the generation of the following weighted adjacency matrices.

$$W_u = Norm(WW^T), \;\; W_i = Norm(W^T W), \tag{16}$$

where $Norm(.)$ represents the normalization function, $W_u \in \mathbb{R}^{|U| \times |U|}$ denotes the probability of co-occurrence of user nodes based on structure, and $W_i \in \mathbb{R}^{|I| \times |I|}$ represents the probability of co-occurrence of item nodes based on structure. Considering that the distance between homogeneous nodes is greater than 2 hops, we use W^U and W^I to represent the co-occurrence probability of a node with its multi-hop neighbors.

$$W^U = \sum_{h=1}^{\gamma} {W_u}^h,\ W^I = \sum_{h=1}^{\gamma} {W_i}^h, \tag{17}$$

where γ is the maximum number of hops between homogeneous nodes that we consider. W^U and W^I represent the empirical probability of a node co-occurring with its homogeneous neighbors. The reconstructed distributions between homogeneous nodes via node representations are defined as:

$$\hat{W}^U_{u,v} = \frac{1}{1 + exp(z_u^T z_v)},\ \hat{W}^I_{i,j} = \frac{1}{1 + exp(z_i^T z_j)}. \tag{18}$$

We achieve the enrichment of representations through structural neighbors by minimizing the KL divergence between two distributions.

$$minimize\ \mathcal{L}_S^U = KL(W^U||\hat{W}^U) = \sum_{W^U_{u,v} \in W^U} [W^U_{u,v} log \frac{W^U_{u,v}}{\hat{W}^U_{u,v}}], \tag{19}$$

$$minimize\ \mathcal{L}_S^I = KL(W^I||\hat{W}^I) = \sum_{W^I_{i,j} \in W^I} [W^I_{i,j} log \frac{W^I_{i,j}}{\hat{W}^I_{i,j}}]. \tag{20}$$

The complete loss of enriching representation through structural neighbors is the sum of the above two losses.

$$minimize\ \mathcal{L}_S = \mathcal{L}_S^U + \mathcal{L}_S^I. \tag{21}$$

3.3.2 Enriching Representation through Semantic Neighbors. Enriching representation through structural neighbors introduces noise due to its reliance on explicit neighbors in interaction graphs. To mitigate this problem, we propose extending contrastive pairs to include semantic neighbors, which are nodes that are not directly connected in the graph but have similar characteristics (item nodes) or preferences (user nodes). We identify semantic neighbors based on the higher similarity of nodes with similar intent distributions. Specifically, we construct user-user and item-item co-occurrence graphs based on the intent distributions of nodes, and generate the following weighted adjacency matrices.

$$M_u = Norm(I^U {I^U}^T),\ M_i = Norm(I^I {I^I}^T), \tag{22}$$

where $Norm(.)$ represents the normalization function, $M_u \in \mathbb{R}^{|U|\times|U|}$ denotes the probability of co-occurrence of user nodes based on semantics, and $M_i \in \mathbb{R}^{|I|\times|I|}$ represents the probability of co-occurrence of item nodes based on semantics. Similar to enriching representation through structural neighbors, we use M^U and M^I to represent the co-occurrence probability of a node with its multi-hop neighbors.

$$M^U = \sum_{h=1}^{\gamma} {M_u}^h,\ M^I = \sum_{h=1}^{\gamma} {M_i}^h, \tag{23}$$

where γ is the maximum number of hops between homogeneous nodes that we consider. We also achieve the enrichment of representations through semantic neighbors by minimizing the KL divergence between two distributions.

$$minimize\ \mathcal{L}_P^U = KL(M^U||\hat{W}^U) = \sum_{M^U_{u,v} \in M^U} [M^U_{u,v} log \frac{M^U_{u,v}}{\hat{W}^U_{u,v}}], \tag{24}$$

$$minimize\ \mathcal{L}_P^I = KL(M^I||\hat{W}^I) = \sum_{M^I_{i,j} \in M^I} [M^I_{i,j} log \frac{M^I_{i,j}}{\hat{W}^I_{i,j}}]. \tag{25}$$

The complete loss of enriching representation through semantic neighbors is the sum of the above two losses.

$$minimize\ \mathcal{L}_P = \mathcal{L}_P^U + \mathcal{L}_P^I. \tag{26}$$

3.4 Optimization

In this section, we will introduce the overall loss and optimize the proposed model using the Expectation-Maximization (EM) algorithm.

Overall Training Objective. In the realm of bipartite graph representation learning, the primary objective is to reconstruct the original graph structure. Building on this foundation, we define an infinite layers propagation constraint to enable node representations to capture global, macroscopic outcomes. Furthermore, we enriching the representations of nodes through both structural and semantic neighbors. This joint optimization process is employed to achieve the optimal representation of nodes.

$$minimize\ \mathcal{L} = \mathcal{L}_R + \lambda_1 \mathcal{L}_I + \lambda_2 \mathcal{L}_S + \lambda_3 \mathcal{L}_P, \tag{27}$$

where λ_1, λ_2, and λ_3 are hyperparameters in learning. We used the Stochastic Gradient Descent (SGD) algorithm to implement the optimization of the joint model to achieve loss minimization and obtain the best representation of the nodes.

Optimization through the EM Algorithm. Since both $\mathcal{L}_I$ and $\mathcal{L}_P$ encapsulate information of intent distribution, which during the learning process is considered a latent variable, we generate this by clustering the representations of all nodes. Therefore, we employ the EM algorithm to optimize the overall loss $\mathcal{L}$.

In the E-step, the node representations of the z_u and z_i are considered observable. The clustering process can be interpreted as solving the following maximum likelihood function:

$$\sum_{u \in U} p(z_u|\Theta, W) = \sum_{u \in U} \sum_{p \in P} p(z_u, z_p|\Theta, W), \tag{28}$$

$$\sum_{i \in I} p(z_i|\Theta, W) = \sum_{i \in I} \sum_{q \in Q} p(z_i, z_q|\Theta, W), \tag{29}$$

where Θ is a set of model parameters and W is the interaction matrix. We utilize Equation (1) to resolve Equations (28) and (29), subsequently deriving the clustering outcomes for nodes of classes U and I, respectively.

In the M-step, we consider the intent distribution of nodes as a known factor and utilize Equation (27) to refine the node representations.

3.5 Model Analysis

In this section, we will compare IDBR with existing representation learning methods.

Comparsion with existing graph convolutional methods. When compared with the graph convolutional models of such as NGCF [39] and LightGCN [14], the intent distribution based convolutional neural network proposed in this paper shifts its focus from aggregating features of neighboring nodes to concentrating on more refined intent points. These intent points are generated through clustering all nodes, and our model incorporates an infinite layer

Table 1: Statistics of Bipartite Networks and Metrics Adopted in Experiments for Different Tasks.

Task	Link Prediction		Recommendation	
Type	unweighted		weighted	
Metric	AUC-ROC,AUC-PR		F1,NDCG,MAP,MRR	
Datasets	Wikipedia	Amazon-Book	DBLP	Movielens-10M
$\lvert U \rvert$	15,000	52,643	6,001	69,878
$\lvert V \rvert$	3,214	91,599	1,308	10,677
$\lvert E \rvert$	172,426	2,984,108	29,256	10,000,054
$Density$	0.40%	0.06%	0.40%	1.30%

Table 2: Link Prediction Performance on Wikipedia and Amazon-Book.

Algorithm	Wikipedia		Amazon-Book	
	AUC-ROC	AUC-PR	AUC-ROC	AUC-PR
Node2vec	89.93%	91.23%	68.87%	69.45%
LINE	91.26%	93.28%	71.32%	72.21%
Metapath2vec++	90.84%	92.42%	70.47%	71.87%
BiNE	92.91%	94.45%	72.23%	74.28%
NGCF	91.83%	93.77%	71.04%	73.67%
IGCN	92.31%	93.50%	71.86%	73.34%
NCL	92.21%	93.40%	71.65%	73.22%
SimRec	92.93%	94.23%	72.20%	74.15%
IDBR	**94.90%**	**96.58%**	**72.85%**	**75.22%**
Improv.	2.12%	2.26%	0.86%	1.27%

propagation constraint. This enables IDBR to capture more comprehensive and macro-level intent information during the aggregation process. Moreover, when constructing the intent distribution, we meticulously consider the weight relations of all paths between nodes and intent points. This consideration significantly reduces the impact of individual interactions on the model's performance, thereby enhancing the effectiveness of our model in both sparse and noisy datasets.

Comparison with existing contrastive learning methods. Compared to existing contrastive learning models like NCL [24],and SimRec [42] our approach not only identifies suitable positive pairs for contrastive learning, but also delves into the intensity relations between these pairs. Additionally, we employ the KL divergence to align the representations of different nodes. This method offers the advantage of using list-wise ranking algorithms [44] to simultaneously handle the alignment of multiple node pairs. Moreover, it avoids the need for negative sampling, thereby shortening the training cycles of the model. These innovations enhance the efficiency and precision of our model, particularly when dealing with complex datasets.

4 EXPERIMENTS

To validate the effectiveness of IDBR, we conduct experiments on four popular datasets to verify the following questions:

RQ1: What is the performance of IDBR in typical task scenarios such as link prediction and recommendations?
RQ2: Does every part of IDBR improve its performance?
RQ3: How is the robustness of IDBR on sparse and noisy datasets?
RQ4: What is the performance effect of adjusting the model's primary hyperparameters?

4.1 Experimental Setting

4.1.1 Experimental tasks and datasets. In this paper, we use the link prediction and recommendation tasks to evaluate the performance of our model. Based on the work of Gao et al. [10], we implemented the link prediction task using unweighted networks, which is typically a classification task to predict links between two nodes, and the recommendation task using weighted networks is a personalized ranking task aimed at providing users with items of interest. Below we illustrate the experimental data.

- **Unweighted bipartite networks.** We constructed two unweighted bipartite networks based on the Wikipedia[1] and Amazon-Book[2] datasets, respectively. The Wikipedia dataset contains editorial relations between authors and pages; the Amazon-Book dataset contains information about users' purchases with books.
- **Weighted bipartite networks.** We constructed two weighted bipartite networks based on the DBLP[3] and Movielens-10M[4] datasets, respectively. The DBLP dataset contains publication relations between authors and venues, where edge weights indicate the number of papers published by an author at a venue; the Movielens dataset records users' rating behavior for movies, where edge weights indicate users' ratings for movies.

Table 1 details the tasks, evaluation metrics, and datasets, where T denotes the type of the bipartite network, $|U|$ is the number of nodes in U, $|V|$ is the number of nodes in V, $|E|$ is the number of edges, and $Density = \frac{|E|}{|U| \times |V|}$.

4.1.2 Evaluation Metrics. For both the link prediction and recommendation tasks, 60% of the data was randomly chosen as the training set, with the remaining 40% used for testing. The performance of link prediction was assessed using the ROC curve (AUC-ROC) and precision-recall curve (AUC-PR) metrics, while in recommendation task, we used the F1@10, NDCG@10, MAP@10, and MRR@10 metrics.

4.1.3 Baselines. In addition to IDBR, we implemented the following state-of-the-art methods as baselines. We classify these baselines into two categories: relation reconstruction and neighbor aggregation.

Relation reconstruction:

- Node2vec[12]: This method extends DeepWalk by employing BFS and DFS strategies during random walks to capture homogeneity and structural features between nodes.
- LINE [35]: This method proposes using first-order and second-order similarities to measure the similarities between nodes.
- metapath2vec++ [8]: This method uses meta-path-based wandering to capture similarities between nodes.
- BiNE [10]: This method optimizes both explicit and implicit relations in a bipartite network to learn node representations.

Neighbor aggregation:

- NGCF [39]: This method forms a network from user-item interactions and employs GCN layers to understand complex user-item connections more effectively.

[1] https://github.com/clhchtcjj/BiNE/tree/master/data/wiki
[2] https://github.com/xue-pai/UltraGCN/tree/main/data/AmazonBooks_m1
[3] http://dblp.uni-trier.de/xml
[4] https://grouplens.org/datasets/movielens/10m/

- IGCN [41]: This method proposes a novel incremental temporal convolutional network for efficient and time-aware feature learning by merging past and present cycle information.
- NCL [24] This method enhances node representations by mining neighbors from graph structures and semantic spaces as positive pairs for contrastive learning.
- SimRec [42] This method introduces a model that adapts contrastive knowledge distillation from a GNN teacher to a compact network, enhancing recommendation efficiency and robustness.

4.1.4 Hyperparameter Setting. We implemented IDBR using PyTorch and used the SGD optimizer during training, where the learning rate was set to 0.1 and the momentum was set to 0.9. We set the value range of weights $\mathcal{L}_1$, $\mathcal{L}_2$, and $\mathcal{L}_3$ between 0.0001 and 1, the neighbor node range parameter γ between 1 and 6, the number of cluster centers between 64 and 160, and the vector dimensions between 32 to 256. For a detailed study of hyperparameters, please refer to Section 4.2.4.

4.2 Experimental Results and Analysis

4.2.1 Performance comparison. Table 2 compares the performance of various algorithms in link prediction, where our algorithm outperforms the current bipartite network representations. Compared with the best existing results, IDBR achieves average improvements of 1.49% in AUC-ROC and 1.77% in AUC-PR, respectively. Similarly, Table 3 shows the advantage of our algorithm in the recommendation task, showing average enhancements of 11.63%, 11.53%, 6.77%, and 9.45% in F1@10, NDCG@10, MAP@10, and MRR@10, respectively, compared with the best existing results.

The reasons for the good performance of IDBR are as follows: (1) Compared with traditional methods, we employ an intent distribution based convolutional neural network to generate node representations. Unlike traditional methods that can only capture local structural information of the graph, our approach can generate macroscopic intent representations of nodes. Additionally, when constructing the intent distribution, we have effectively addressed the problems of data sparsity and noise. (2) We consider the strength of connections and distances between nodes, exploring similarities in both structure and semantics with neighbors to create constraints that enrich node representations.

4.2.2 Ablation studies. In this section, we validate the effectiveness of each module in the IDBR model through three variants, using parameters that yield the best results for the model.

- **w/o I:** Eliminate the intent distribution based graph convolution neural network module.
- **w/o S:** Eliminate the enriching representation with structural neighbors module.
- **w/o P:** Eliminate the enriching representation through semantic neighbors module.

The results of the ablation studies are presented in Tables 4 and 5, focusing only on the DBLP and Wikipedia datasets. Similar outcomes were observed in other datasets. From Tables 4 and 5, it is evident that: (1) Eliminating the intent distribution based graph convolution neural network module leads to a significant decline in the algorithm's performance. (2) Eliminating the enriching representation through structural and semantic neighbors also impacts the algorithm's effectiveness.

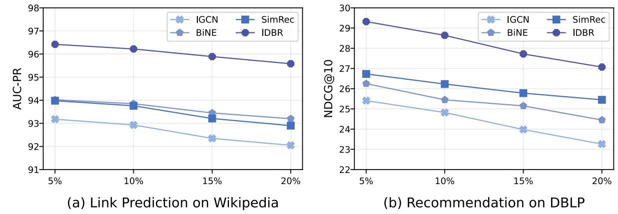

Figure 5: Performance Comparison on Sparse Datasets.

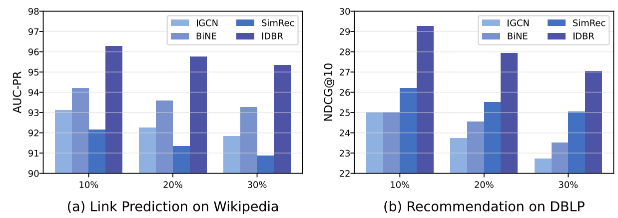

Figure 6: Performance Comparison on Noisy Datasets.

The reasons for such ablation results are as follows: (1) We obtain the nodes' intent distributions through a global clustering algorithm. The intent distribution based convolution module captures the global intent information of nodes. Without it, nodes can only access local structural information. Using intent distribution convolution can achieve purer and more macroscopic node representations, thus its removal significantly impacts the algorithm's performance. (2) The enriching representation through both structural and semantic neighbors provides additional supervised signals during learning, which make node representations closer to the original graph structure. Consequently, eliminating these modules also affects the algorithm's performance.

4.2.3 Robustness Analysis. In this section, we validate the robustness of IDBR by assessing its performance on sparse and noisy datasets.

Performance on Sparse Datasets. In the process of bipartite graph representation learning, the density of the graph's structure significantly impacts the algorithm's performance. Therefore, our research aims to demonstrate the effectiveness of our model in addressing the issue of data sparsity issues. To illustrate this, we randomly removed edges (5%, 10%, 15%, and 20%) from the original dataset and then tested our model's effectiveness on sparse datasets for link prediction and recommendation tasks. Due to space constraints, for the link prediction task, we only present the results for the AUC-PR metric on the Wikipedia dataset. For the recommendation task, only the results of the NDCG@10 metric on the DBLP dataset are shown. Similar results were observed for other metrics and datasets. The performance on sparse datasets is illustrated in Figure 5.

Figure 5 demonstrates that the IDBR model consistently performs well on datasets with varying levels of sparsity. This is attributed to our method of determining node intent distributions through

Table 3: Top-10 recommended performance comparison on DBLP and Movielens-10M.

Algorithm	DBLP				Movielens-10M			
	F1@10	NDCG@10	MAP@10	MRR@10	F1@10	NDCG@10	MAP@10	MRR@10
Node2vec	8.54%	23.89%	19.44%	31.11%	4.16%	3.68%	1.05%	18.33%
LINE	8.99%	14.41%	9.62%	17.13%	6.91%	6.50%	1.74%	38.12%
Metapath2vec++	8.65%	25.14%	19.06%	31.97%	4.65%	4.39%	1.91%	36.60%
BiNE	11.37%	26.19%	20.47%	33.36%	9.14%	9.02%	3.01%	45.95%
NGCF	10.57%	25.58%	19.51%	32.57%	9.68%	10.07%	3.97%	47.52%
IGCN	11.29%	25.91%	20.30%	33.41%	10.10%	9.65%	3.62%	48.33%
NCL	11.43%	24.84%	20.31%	32.53%	9.73%	9.21%	3.90%	46.40%
SimRec	12.54%	26.89%	21.44%	33.11%	10.16%	10.68%	4.15%	48.32%
IDBR	**13.83%**	**30.32%**	**23.31%**	**38.60%**	**11.46%**	**11.78%**	**4.35%**	**49.96%**
Improv.	10.45%	12.76%	8.72%	15.53%	12.80%	10.30%	4.82%	3.37%

Table 4: Ablation study for link prediction (Wikipedia).

Algorithm	Metrics	
	AUC-ROC	AUC-PR
w/o I	93.55%	95.01%
w/o S	94.03%	95.78%
w/o P	94.44%	95.92%
IDBR	94.90%	96.58%

Table 5: Ablation study for recommendation (DBLP).

Algorithm	Metrics			
	F1@10	NDCG@10	MAP@10	MRR@10
w/o I	11.58%	26.65%	20.86%	33.72%
w/o S	11.98%	28.16%	22.57%	35.38%
w/o P	12.22%	29.27%	23.03%	36.35%
IDBR	13.83%	30.32%	23.31%	38.60%

matrix operations, where the distribution is calculated based on all neighbors of a node, minimizing the impact of individual data point omissions. Additionally, our enriching representation approach explores the relations between nodes and their multi-hop neighbors, including both structural and semantic relations, effectively mitigating the problem of data sparsity.

Performance on Noisy Datasets. In bipartite graph representation learning, noise significantly impacts the results. Hence, our study focuses on the model's performance with noisy data. To simulate real-world uncertainties and their effects on the bipartite graph representation, we replaced real edges with (10%, 20%, and 30%) noise in our experiments. For the link prediction task, we primarily show the AUC-PR results on the Wikipedia dataset. For the recommendation task, we focus on the NDCG@10 results on the DBLP dataset. Similar results were observed with other metrics and datasets. The effects on noisy datasets are shown in Figure 6.

From Figure 6, it is evident that the IDBR model consistently performs well on noisy datasets. This is because the intent distribution of a node is calculated based on all its neighbors, thus minimizing the impact of individual noise. Furthermore, when exploring the relations between nodes and their neighbors, we take into account all path relations between them, reducing the susceptibility of overall relation exploration to minor noise.

4.2.4 Hyperparameter Study. In this section, we assess the impact of weights $\mathcal{L}_1$, $\mathcal{L}_2$, and $\mathcal{L}_3$, the neighbor range parameter γ, the number of cluster centers, and the vector dimension d on the

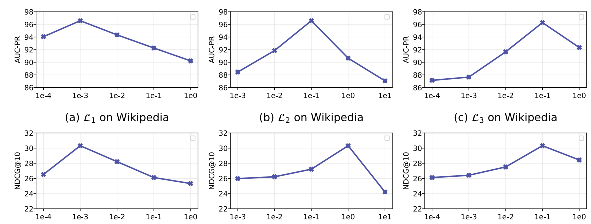

Figure 7: Impact of the weights on the model's performance.

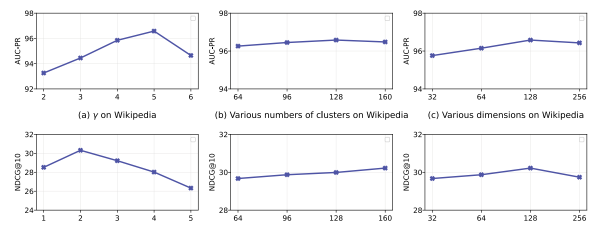

Figure 8: Impact of the neighbor range, the number of cluster centers, and the vector dimension on the model's performance.

model's performance. We only present experimental results exclusively from Wikipedia and DBLP datasets, similar results can be obtained from other datasets as well. The experimental results showing the variation of weights on the Wikipedia and DBLP datasets are presented in Figure 7, while the experimental results showing the variation of neighbor range, number of cluster centers, and vector dimensions are presented in Figure 8.

Impact of the weights $\mathcal{L}_1$, $\mathcal{L}_2$, and $\mathcal{L}_3$. To validate the influence of varying weights on the performance of IDBR, we tuned the ranges of $\mathcal{L}_1$, $\mathcal{L}_2$, and $\mathcal{L}_3$ from 0.0001 to 1, with the experimental results shown in Figure 7. The results reveal that the optimal performance for the link prediction task on the Wikipedia dataset is achieved when $\mathcal{L}_1 = 0.001$, $\mathcal{L}_2 = 0.1$, and $\mathcal{L}_3 = 0.1$. In contrast, the best results are obtained for the recommendation task on the DBLP dataset when $\mathcal{L}_1 = 0.001$, $\mathcal{L}_2 = 1$, and $\mathcal{L}_3 = 0.1$. This indicates that when handling the DBLP dataset, there is a greater emphasis on enriching representation through structural neighbors. This is due to DBLP's weighted graph structure, which allows for a more precise extraction of neighbor relation information, thereby justifying its higher weight.

Impact of the neighbor range parameter γ. To explore the impact of the neighbor range on the performance of IDBR, we tuned the range of γ from 1 to 6, with the results presented in Figure 8(a) and Figure 8(d). The findings reveal that for the link prediction task on the Wikipedia dataset, the optimal result is achieved when $\gamma = 5$, while for the recommendation task on the DBLP dataset, the best result occurs when $\gamma = 2$. This difference arises because Wikipedia, as an unweighted graph, requires longer paths to determine the relations between nodes. In contrast, DBLP, as a weighted graph, benefits from shorter paths, as overly extended paths might lead to a deviation in the relations between nodes.

Impact of the number of cluster centers. We tuned the number of cluster centers from 64 to 160 to analyze its impact on performance. The experimental results are displayed in Figure 8(b) and Figure 8(e). The optimal performance is achieved for the link prediction and recommendation tasks with 128 clusters, while the cluster size has a minimal impact on performance. This is because the model can consistently identify key clusters with strong associations, regardless of the total number of clusters, ensuring stable performance despite variations.

Impact of the vector dimension d. To analyze the impact of the vector dimensions on IDBR's performance, we tuned the range of d from 32 to 256, with the experimental results displayed in Figure 8(c) and Figure 8(f). It is observed that IDBR achieves its peak performance in the link prediction and recommendation tasks when $d = 128$. Additionally, the vector dimension has a limited effect on the model, yielding favorable outcomes across other dimensions as well. This fully demonstrates the robustness of the model.

4.2.5 Summary. The above experiments illustrate that IDBR is an effective bipartite network representation learning algorithm. We confirmed the effectiveness of the algorithm in the link prediction and recommendation tasks, respectively. On average, we achieved a 1.30% performance improvement in the link prediction and a 9.84% performance improvement in the recommendation task. We validated the effectiveness of our algorithm's components through ablation studies. The robustness of our algorithm has been validated through experiments conducted on sparse and noisy datasets. Moreover, a thorough analysis of hyperparameters was performed to determine the optimal conditions for achieving the best results of our model.

5 RELATED WORK

5.1 Bipartite graph representation learning

The existing bipartite graph representation learning approaches fall into two primary categories: relation reconstruction and neighborhood aggregation. Relation reconstruction aims to reconstruct the structural relations between nodes in the graph using their node representations. Several studies [8, 10, 12] use random walks and the Skip-gram method to learn node vector representations. For example, Node2vec [12] enhanced this approach by focusing on the node homogeneity and structure. BiNE [10] optimizes both explicit and implicit relations in a bipartite network to learn node embeddings. PinSAGE [43] and RecWalk [28] combined advanced random walks with graph neural networks for more effective learning. Neighbor aggregation methods generate their own representations by iteratively aggregating information from their neighbors. For example, GCN [16], GraphSAGE [13], and GAT [37] focus on local structures around nodes, employing techniques such as graph convolution, multi-layer aggregation, and attention mechanisms, respectively. NGCF [39] uses a bipartite network to propagate embeddings, focusing on high-order network connectivity and integrating collaborative signals. LightGCN [14] simplifies NGCF [39] by linearly propagating user and item embeddings and combining all layer embeddings for the final output. Models such as VAGE [17], GC-MC [1], and Multi-GCCF [34] have adopted complex network encoders and graph autoencoder frameworks for better user-item interaction simulation.

5.2 Intent information in representation learning

In bipartite graphs, user-item interactions implicitly reflect their intentions. Recent research highlights the significance of user intent in improving representation learning [3, 19, 20, 30, 33, 38]. For example, MCPRN [38] models items using mixed-channel methods for multiple purposes. MIND [19], inspired by capsule networks, learns diverse user interests through dynamic routing. ComiRec [30] captures various interests from user behavior, aggregating them for comprehensive recommendations. Pan et al. [38] introduced an intent-guided method to identify and integrate relevant session neighbors. Ma et al. [26] focus on deciphering users' intents to form related sample pairs. The ASLI method [36] employs a temporal convolutional network to extract latent intents. Chang et al. [4] use a Variational Autoencoder (VAE) to infer user intent as a latent variable. IOCRec [21] using intent-level high-quality views to reduce noise in contrastive SR models. The ICLRec framework [7] employs contrastive learning and clustering to learn intent variables, maximizing the mutual information among similar users.

6 CONCLUSIONS

In this paper, we propose the Intent Distribution based Bipartite graph Representation learning (IDBR) model. In IDBR, we have developed an intent distribution based graph convolutional neural network that aggregates intention information obtained through clustering operations. Unlike traditional bipartite graph representations, IDBR captures macroscopic intent information instead of local structure. We've effectively addressed data sparsity and noise when constructing node intent distributions. Furthermore, we also separately explore the similarity relations between nodes and their neighbors from both structural and semantic perspectives, using these as constraints to enrich node representations. Our link prediction and recommendation experiments indicate that the model has superior performance.

7 ACKNOWLEDGMENTS

This work is sponsored by the National Natural Science Foundation of China (Nos. 61973180, 62172249, 62202253), the Natural Science Foundation of Shandong Province (Nos. ZR2021MF092, ZR2021QF074, ZR2019MF014, ZR2019MF033, ZR2022MF326) and the China Scholarship Council (Grant No. 202308370301).

REFERENCES

[1] Rianne van den Berg, Thomas N Kipf, and Max Welling. 2017. Graph convolutional matrix completion. *arXiv preprint arXiv:1706.02263* (2017).

[2] Jiangxia Cao, Xixun Lin, Shu Guo, Luchen Liu, Tingwen Liu, and Bin Wang. 2021. Bipartite graph embedding via mutual information maximization. In *WSDM*. 635–643.

[3] Yukuo Cen, Jianwei Zhang, Xu Zou, Chang Zhou, Hongxia Yang, and Jie Tang. 2020. Controllable multi-interest framework for recommendation. In *KDD*. 2942–2951.

[4] Bo Chang, Alexandros Karatzoglou, Yuyan Wang, Can Xu, Ed H Chi, and Minmin Chen. 2023. Latent user intent modeling for sequential recommenders. In *WWW*. 427–431.

[5] Chih-Ming Chen, Chuan-Ju Wang, Ming-Feng Tsai, and Yi-Hsuan Yang. 2019. Collaborative similarity embedding for recommender systems. In *WWW*. 2637–2643.

[6] Hongxu Chen, Hongzhi Yin, Tong Chen, Weiqing Wang, Xue Li, and Xia Hu. 2020. Social boosted recommendation with folded bipartite network embedding. *IEEE Transactions on Knowledge and Data Engineering* 34, 2 (2020), 914–926.

[7] Yongjun Chen, Zhiwei Liu, Jia Li, Julian McAuley, and Caiming Xiong. 2022. Intent contrastive learning for sequential recommendation. In *WWW*. 2172–2182.

[8] Yuxiao Dong, Nitesh V Chawla, and Ananthram Swami. 2017. metapath2vec: Scalable representation learning for heterogeneous networks. In *KDD*. 135–144.

[9] Chen Gao, Yu Zheng, Nian Li, Yinfeng Li, Yingrong Qin, Jinghua Piao, Yuhan Quan, Jianxin Chang, Depeng Jin, Xiangnan He, et al. 2023. A survey of graph neural networks for recommender systems: Challenges, methods, and directions. *ACM Transactions on Recommender Systems* 1, 1 (2023), 1–51.

[10] Ming Gao, Xiangnan He, Leihui Chen, Tingting Liu, Jinglin Zhang, and Aoying Zhou. 2022. Learning Vertex Representations for Bipartite Networks. *IEEE Transactions on Knowledge and Data Engineering* 34, 1 (2022), 379–393.

[11] Thomas Gaudelet, Ben Day, Arian R Jamasb, Jyothish Soman, Cristian Regep, Gertrude Liu, Jeremy BR Hayter, Richard Vickers, Charles Roberts, Jian Tang, et al. 2021. Utilizing graph machine learning within drug discovery and development. *Briefings in bioinformatics* 22, 6 (2021), bbab159.

[12] Aditya Grover and Jure Leskovec. 2016. Node2vec: Scalable feature learning for networks. In *KDD*. 855–864.

[13] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).

[14] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *SIGIR*. 639–648.

[15] Junzhong Ji, Hao Jia, Yating Ren, and Minglong Lei. 2023. Supervised Contrastive Learning with Structure Inference for Graph Classification. *IEEE Transactions on Network Science and Engineering* (2023).

[16] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[17] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016).

[18] Lecheng Kong, Yixin Chen, and Muhan Zhang. 2022. Geodesic graph neural network for efficient graph representation learning. *Advances in Neural Information Processing Systems* 35 (2022), 5896–5909.

[19] Chao Li, Zhiyuan Liu, Mengmeng Wu, Yuchi Xu, Huan Zhao, Pipei Huang, Guoliang Kang, Qiwei Chen, Wei Li, and Dik Lun Lee. 2019. Multi-interest network with dynamic routing for recommendation at Tmall. In *CIKM*. 2615–2623.

[20] Haoyang Li, Xin Wang, Ziwei Zhang, Jianxin Ma, Peng Cui, and Wenwu Zhu. 2021. Intention-aware sequential recommendation with structured intent transition. *IEEE Transactions on Knowledge and Data Engineering* 34, 11 (2021), 5403–5414.

[21] Xuewei Li, Aitong Sun, Mankun Zhao, Jian Yu, Kun Zhu, Di Jin, Mei Yu, and Ruiguo Yu. 2023. Multi-Intention Oriented Contrastive Learning for Sequential Recommendation. In *WSDM*. 411–419.

[22] Zhao Li, Xin Shen, Yuhang Jiao, Xuming Pan, Pengcheng Zou, Xianling Meng, Chengwei Yao, and Jiajun Bu. 2020. Hierarchical bipartite graph neural networks: Towards large-scale e-commerce applications. In *ICDE*. 1677–1688.

[23] David Liben-Nowell and Jon Kleinberg. 2003. The link prediction problem for social networks. In *CIKM*. 556–559.

[24] Zihan Lin, Changxin Tian, Yupeng Hou, and Wayne Xin Zhao. 2022. Improving graph collaborative filtering with neighborhood-enriched contrastive learning. In *WWW*. 2320–2329.

[25] Linyuan Lü and Tao Zhou. 2011. Link prediction in complex networks: A survey. *Physica A: statistical mechanics and its applications* 390, 6 (2011), 1150–1170.

[26] Jianxin Ma, Chang Zhou, Hongxia Yang, Peng Cui, Xin Wang, and Wenwu Zhu. 2020. Disentangled self-supervision in sequential recommenders. In *KDD*. 483–491.

[27] Kelong Mao, Jieming Zhu, Xi Xiao, Biao Lu, Zhaowei Wang, and Xiuqiang He. 2021. UltraGCN: ultra simplification of graph convolutional networks for recommendation. In *CIKM*. 1253–1262.

[28] Athanasios N Nikolakopoulos and George Karypis. 2019. Recwalk: Nearly uncoupled random walks for top-n recommendation. In *WSDM*. 150–158.

[29] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).

[30] Zhiqiang Pan, Fei Cai, Yanxiang Ling, and Maarten De Rijke. 2020. An Intent-guided Collaborative Machine for Session-based Recommendation. In *SIGIR*. 1833–1836.

[31] Franz Pernkopf and Djamel Bouchaffra. 2005. Genetic-based EM algorithm for learning Gaussian mixture models. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 27, 8 (2005), 1344–1348.

[32] Xiuyuan Qin, Huanhuan Yuan, Pengpeng Zhao, Junhua Fang, Fuzhen Zhuang, Guanfeng Liu, Yanchi Liu, and Victor Sheng. 2023. Meta-optimized contrastive learning for sequential recommendation. In *SIGIR*. 89–98.

[33] Xiuyuan Qin, Huanhuan Yuan, Pengpeng Zhao, Guanfeng Liu, Fuzhen Zhuang, and Victor S Sheng. 2024. Intent Contrastive Learning with Cross Subsequences for Sequential Recommendation. In *WSDM*. 548–556.

[34] Jianing Sun, Yingxue Zhang, Chen Ma, Mark Coates, Huifeng Guo, Ruiming Tang, and Xiuqiang He. 2019. Multi-graph convolution collaborative filtering. In *ICDM*. 1306–1311.

[35] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. 2015. Line: Large-scale information network embedding. In *WWW*. 1067–1077.

[36] Md Mehrab Tanjim, Congzhe Su, Ethan Benjamin, Diane Hu, Liangjie Hong, and Julian McAuley. 2020. Attentive sequential models of latent intent for next item recommendation. In *WWW*. 2528–2534.

[37] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).

[38] Shoujin Wang, Liang Hu, Yan Wang, Quan Z Sheng, Mehmet Orgun, and Longbing Cao. 2019. Modeling multi-purpose sessions for next-item recommendations via mixture-channel purpose routing networks. In *IJCAI*.

[39] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *SIGIR*. 165–174.

[40] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *WWW*. 2022–2032.

[41] Jiafeng Xia, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, and Ning Gu. 2021. Incremental graph convolutional network for collaborative filtering. In *CIKM*. 2170–2179.

[42] Lianghao Xia, Chao Huang, Jiao Shi, and Yong Xu. 2023. Graph-less collaborative filtering. In *WWW*. 17–27.

[43] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *KDD*. 974–983.

[44] Xiangyu Zhao, Liang Zhang, Long Xia, Zhuoye Ding, Dawei Yin, and Jiliang Tang. 2017. Deep reinforcement learning for list-wise recommendations. *arXiv preprint arXiv:1801.00209* (2017).

[45] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI open* 1 (2020), 57–81.